# Effect of Visual Scene Complexity on Deep Reinforcement Learning

Travis Boyd

***Abstract*—I wanted to test if having a more complex scene affects the learning speed of a deep reinforcement learning agent which was trained only using image observations. I used glitter, an open source repository that links GLFW, GLAD, Bullet3 and other libraries to create basic basketball shooting simulator. With this simulator, I set up three scene configurations: low, medium, and high complexity scenes. The complexity of the scene was measured by the textures and lighting used within that scene. A Proximal Policy Optimization (PPO) agent with a convolutional neural network (CNN) observation encoder was trained on each of the configurations. Originally, I had the agent shoot from a single position, but this allowed the agent to just find the right parameters to score a basket without actually observing the scene, so ball spawn positions were randomized for every episode. I also implemented a two phase approach to training where the ball position was fixed for the first 25% of the training so that the model would understand how to score a basket and then for the last 75% of training the ball positions would be randomized.**

***Index Terms*—Reinforcement Learning, Proximal Policy Optimization, Visual Complexity, Curriculum Learning, Physics Simulation, Convolutional Neural Network**

## I. Introduction

Deep reinforcement learning (RL) agents are able to use raw pixel observations to achieve good results. For example, an RL agent was able to play the Atari [1]. This is due to the fact that CNNs are good at extracting task relevant features from visual inputs. However, the complexity of a visual scene could influence how efficiently a CNN feature extractor can get the details that matter for a specific task. Things that make a scene visual complex include things like surface textures, color, and lighting. An agent that tries to aim a basketball in a scene that only contains blank walls and a white backboard will have less to process than an agent inside of a fully textured gym scene with multiple sources of lighting.

For this project, the focus is to see whether or not increasing the complexity of a scene affects the learning of a PPO agent that has to use image observations to make its decisions. Prior work on this topic in RL has been done in this paper [2] which demonstrated that irrelevant visual information can slow convergence. In this paper, they used synthetic background noise to add complexity to the scene, but for my project I wanted to use natural scene complexity.

To answer this question, a basketball shooting simulator built in C++ and OpenGL with physics from Bullet3 was created. The scene contains court with a hoop, backboard, and a net. There are three scenes: low, medium, and high complexity. A PPO agent with a CNN policy get a $84 \times 84$ RGB image rendered from the balls position and must choose a shot based on yaw, pitch, and power to make the shot.

A big problem with this project was getting the agent to actually use the images rather than just memorizing an action that makes the ball into the basket. To fix this, the balls starting spot is randomized each episode over a $5 \times 0.5 \times 3\,\mathrm{m}$ volume in front of the basketball hoop. Because the optimal pitch and power depend on the distance and height to the hoop the image observation is the only data available to the agent.

Training the RL agent from random spawns was taking a long time, so a two phase training cycle was used. In phase 1 the agent trains on a fixed start position for the first 25% of the training. This gives phase 2 a good starting point because in phase 2 the balls position is randomized. So basically, phase 1 agent learns how to make a shot, and phase 2 agent learns how to make a shot based on visual observations.

## II. Related Work

### A. Deep Reinforcement Learning from Pixels

In a paper by Mnih et al. [1], it was proved that a single deep Q-network could learn to play a bunch of different Atari 2600 games by using $84 \times 84$ grayscale screen pixels. The performance of this model matched and exceeded humans on many of the games.

### B. Proximal Policy Optimization

Proximal Policy Optimization (PPO) [3] is an on policy actor critic algorithm that makes each policy update constricted by using a clipped surrogate objective, and preventing large policy changes. For this project, I used the stable baselines3 implementation [4], which has PPO with the NatureCNN feature extractor [1] for image observations.

### C. Visual Complexity and Distractors in RL

Zhang et al. [2] showed that agents trained on observations changed with natural video backgrounds generalize poorly to new backgrounds, which shows that CNNs can overfit to irrelevant visual features. Cobbe et al. [5] showed that visual diversity is necessary for generalization. This is because agents trained on limited visual variety overfit and fail on levels they have not seen. The work for this project kind of complements both because this project studies how the inherent complexity of a fixed environment affects the rate and quality of learning instead of studying the generalization across different environments.

## III. Environment Design

### A. Overview

I created a real time 3D basketball shooting simulator with C++, OpenGL, GLFW, GLAD, and Bullet3. The scene is supposed to look like an indoor basketball court that is $12 \times 12 \times 6$ m. It has a floor, four walls, a ceiling, a backboard, and a hoop. The basketball with realistic material properties is spawned in from a configurable spawn position.

The scene is shown in Fig. 1.

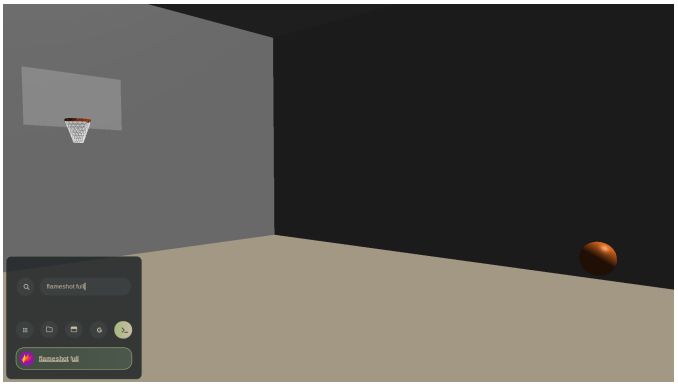

Fig. 1. The basketball environment (low complexity). The agent sees a $84 \times 84$ RGB image rendered from the ball's spawn position, looking toward the hoop.

### B. Physics

All the physics is handled by Bullet3 [6]. The basketball is a dynamic rigid sphere with a radius $r = 0.12$ m, mass $m = 0.624$ kg, and restitution $e = 0.76$. Continuous collision detection (CCD) is enabled on the ball to prevent it from going through the rim. My initial training attempts had the ball complete just pass through the rim and hit the target. This simulation runs at 120 Hz to make sure that the rim and ball collisions are actually present.

### C. Visual Complexity Configurations

There are JSON scene configurations that handle the surface textures and lighting to have different levels of visual complexity. The three levels are low, medium, and high.

Table I summary of the configurations.

TABLE I
Scene Configuration Summary

| Property | Low | Medium | High |
|---|---|---|---|
| Floor texture | none (solid) | wood | parquet |
| Wall texture | none (solid) | stone | brick |
| Ceiling texture | none (solid) | none | plaster |
| Point lights | 0 | 1 | 3 |
| Ambient level | 0.15 | 0.20 | 0.30 |

### D. Visual Complexity Metric

To actually get how complex the scene is, I used two metrics that were computed from $n = 16$ reset state observations per configuration.

Sobel edge density is the mean gradient magnitude computed by applying horizontal and vertical Sobel filters to a grayscale version of each frame:

$$E = \frac{1}{HW} \sum_{i,j} \sqrt{G_x(i,j)^2 + G_y(i,j)^2}, \tag{1}$$

where $G_x$ and $G_y$ are the horizontal and vertical Sobel responses. When there is a higher edge density, this means that there are more fine grained details in the observation. Pixel entropy is the Shannon entropy of the grayscale pixel intensity histogram:

$$H = -\sum_{k=0}^{255} p_k \log_2 p_k \quad \text{(bits)}, \tag{2}$$

where $p_k$ is the empirical probability of intensity $k$. Higher entropy means there is a broader and more unifrom intensity distribution that means there is more variation in the surface color and shading.

A combined complexity score is computed as the normalized average of both metrics:

$$C = \frac{1}{2}\left(\frac{E}{0.15} + \frac{H}{8}\right), \tag{3}$$

where the denominators approximate the practical maxima for typical OpenGL scenes rendered at $84 \times 84$ resolution. Fig. 2 shows representative observations from each configuration alongside their complexity scores.

Low (0.394) Medium (0.888) High (0.944)

Fig. 2. Representative $84 \times 84$ agent observations from the low (left), medium (centre), and high (right) complexity configurations. Combined complexity scores are shown beneath each image.

### E. Spawn Randomization

In each episode the ball is placed at a random position:

$$x \sim \mathcal{U}(-2.5,\ 2.5)\,\text{m},$$
$$y \sim \mathcal{U}(1.0,\ 1.5)\,\text{m},$$
$$z \sim \mathcal{U}(2.5,\ 5.5)\,\text{m}.$$

The agents camera is a the spawn position but above it by 0.6 m and made to look at the hoop. This makes sure that the hoop is always visible but the hoop will have different scales and positions based on the distance and its position horizontally. In the future, I would like to program the camera such that it moves until it spots the hoop.

## IV. METHODS

### A. RL Formulation

The task for this project is a one step Markov Decision Process (MDP). Each episode has one decision. The agent looks at an $84 \times 84$ RGB image $s_t$ and selects an action $a_t \in [-1, 1]^3$. This action is decoded into parameters and applied to the physics simulation. The episodes stop right after the ball passes through the hoop or rests on the floor.

The observation space $s_t \in \{0, \dots, 255\}^{84\times84\times3}$ is a color image rendered into an OpenGL Frame Buffer Object (FBO) at $84 \times 84$ resolution from the agent camera.

The action space is the three dimensional continuous action $a_t = (\hat{\delta}_\psi, \hat{\theta}, \hat{v})$ and is decoded by linear interpolation from $[-1, 1]$ to physical units:

- **Yaw offset** $\delta_\psi \in [-25°,\ 25°]$: This is for later implementation, but the agent camera faces the hoop. If the agent camera did have to find the hoop and face it, this would be implemented.
- **Pitch angle** $\theta \in [5°,\ 60°]$: the angle of the launch velocity vector.
- **Launch power** $v \in [5,\ 25]\,\mathrm{m\,s^{-1}}$: the magnitude of the initial velocity.

**Reward function.** The shaped terminal reward is:

$$r = \begin{cases} +1.0 & \text{if the ball passes through the hoop,} \\ -1 + \dfrac{1}{1 + d_{\min}} & \text{otherwise,} \end{cases} \tag{4}$$

where $d_{\min}$ is the minimum distance from the ball center to the hoop center over the entire path of the ball. A ball that grazes the rim receives a reward near $0$, while a ball that flies far off course receives a reward near $-1$. I used this reward system because it gives gradient signals even on missed shots meaning that the agent will get closer to scoring incrementally.

### B. Network Architecture

I used the PPO implementation from Stable-Baselines3 [4] with its default `CnnPolicy`. The observation encoder is from NatureCNN [1]. It has three convolutional layers ($32 \times 8^2$, stride 4; $64 \times 4^2$, stride 2 and $64 \times 3^2$, stride 1) followed by a $512$ unit fully connected layer with ReLU activations. Observations are normalized to $[0, 1]$ getting into the network. There is a linear actor head that gives as output the mean of a diagonal Gaussian policy over the three actions and there is also a separate linear critic head that outputs the state value estimate.

### C. Training Hyperparameters

PPO is trained with the SB3 default hyperparameters except for the entropy coefficient which is set to $\alpha_H = 0.02$. This allows the RL agent to keep exploring throughout the training.

Key hyperparameters are listed in Table II.

TABLE II
PPO HYPERPARAMETERS

| Parameter | Value |
|---|---|
| Algorithm | PPO [3] |
| Policy | CnnPolicy (NatureCNN encoder) |
| Learning rate | $3 \times 10^{-4}$ |
| Rollout buffer size | 2048 steps |
| Minibatch size | 64 |
| Optimisation epochs | 10 per rollout |
| Discount factor | $\gamma = 0.99$ |
| Entropy coefficient | $\alpha_H = 0.02$ |
| Clip range | 0.2 |
| Total steps per config | 500 000 |

### D. Curriculum Learning

Training the agent on a random spawn proved to be bad. The probability that it actually made a good shot from a random positions is very low and did not produce a lot of reward signals. To fix this, I used a two phase approach.

**Phase 1 — Fixed spawn (25% of total steps, 125 000 steps).** In this phase the ball always spawns at this position: $(0, 1.2, 4.5)$ m. This makes the task really easy because the RL agent only has to find the pitch and power to score from a fixed distance. This basically initializes the agents shooting skill without requiring the visual generalization I was looking for.

**Phase 2 — Random spawn (75% of total steps, 375 000 steps).** In this phase, the agent is trained from random spawns. This means that the agent's learned shooting policy needs to generalize the hoop that can appear at different sizes and heights because of the agents position.

`BestModelCheckpoint`: This saves the model weights whenever the success rate (over the last 200 episodes) reaches is a new best success rate. This keeps the best policy seen during training.

### E. Simulator Architecture

For the simulation architecture, a unix socket connects the Python training loop to the C++ simulator. After each reset, the simulator places the ball at its spawn position then renders the $84 \times 84$ RGB observation from the agent camera into an FBO and sends it to Python as a raw byte array. It also sends a zero reward and done = False. After every step the Python side sends the decoded $(yaw, pitch, power)$ to the simulator and applies the velocity to the ball and fast forwards the simulation until the episode is done. After this, it sends its observations to the terminal with the reward and done = true. The physics runs uncapped in headless mode, so each episode is done in microseconds so that hundreds of training steps can happen per second.

## V. EXPERIMENTS & RESULTS

### A. Visual Complexity Scores

Table III shows the complexity metrics for each scene configuration. You can see that the low configuration produced the simplest observations, with low edge density and low

entropy showing the flat solid color scene. The medium configuration has a large jump in both of the metrics because of the new textures. And the high configuration is only slightly more complicated. I think next time I will add things into the environment like objects and different colored lighting.

TABLE III
VISUAL COMPLEXITY METRICS PER CONFIGURATION

| Config | Edge Density | Entropy (bits) | Combined |
|---|---|---|---|
| Low | 0.0664 | 2.756 | 0.394 |
| Medium | 0.1396 | 6.771 | 0.888 |
| High | 0.1794 | 7.107 | 0.944 |

The gap between low (0.394) and medium/high (>0.88) is shows that there is actually a difference in the complexity between low and the medium and high scenes.

### B. Learning Curves and Training Dynamics

Fig. 3 shows the mean episode reward over training for each configuration.

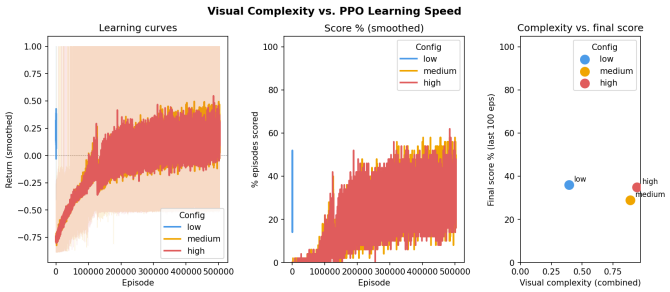


Fig. 3. Mean episode reward over training for medium and high configurations. The vertical dashed line at step 125 000 marks the Phase 1 to Phase 2 curriculum transition. The horizontal reference line shows the mean reward of the pre-trained low model (0.235).

So this is a little skewed, because I ran trained the low and then ran the experiment on accident. If given more time, I would run the experiment again.

Both the medium and high agents did follow the same two phase pattern. In the first phase, the reward rose steadily. When transitioning to the second phase however the reward dipped as the agents had to adapt to new spawn positions. After a second though the agents recovered and even continued to improve. At the end, both the high and medium agents received postive mean rewards which shows that the curriculum actually transferred the shooting skill to the spawn settings.

### C. Final Performance

Table IV summarizes the performance metrics at the end of training. The low configuration was done with a pretrained curriculum because I did not run it in the experiment on accident and ran out of time. The medium and high were trained fresh during the experiment.

The overall success rates for medium and high are lower than the low configuration, but this is because the overall figures averages across the entire training including phase 1.

TABLE IV
FINAL PERFORMANCE PER CONFIGURATION

| Config | Complexity | Overall Success | Last-100 Success | Last-100 Mean Rew |
|---|---|---|---|---|
| Low | 0.394 | 36.1% | 36.0% | +0.233 |
| Medium | 0.888 | 19.8% | 29.0% | +0.142 |
| High | 0.944 | 19.6% | 35.0% | +0.222 |

The last 100 success rates show the performance at the end of training give a good comparison. Low achieved the best performance, and high recieved the middle performance, and medium had the worst performance.

Fig. 4 plots combined complexity score against final success rate.

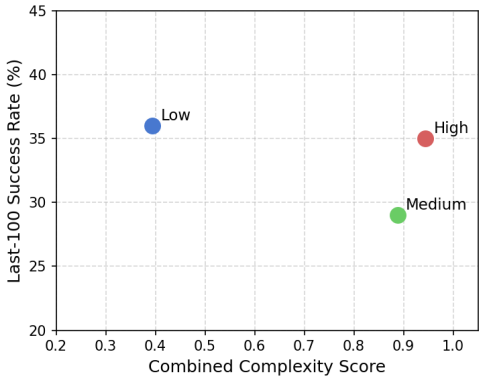


Fig. 4. Visual complexity score vs. final success rate (last-100 episodes) for each configuration.

### D. Discussion

I was really surprised that the agent inside of the high complexity configuration scene matched the low configuration agent in final performance even though the high complexity scene was almost twice as complex based on the metrics I used.

This means that the additional visual information does not impair the CNN when it tries to get relevant features for the task.

The medium configuration performed lower than both. I have no idea why this is. I bet if I ran the experiment again that the medium might score higher. So this probably represents random variation.

Overall, the results show that the scene complexity with the configurations I created does not impede the PPO learning. I believe this has implications for the design of visual RL benchmarks and training environments.

## VI. CONCLUSION

### A. Summary of Findings

This project researched if visual complexity of a 3D rendered sene affects the learning speed and final performance of a PPO agent trained on image observations.

The main thing found was that the visual complexity I implemented did not impair the performance. The high complexity agent got a final success rate of 35% match the low complexity agent who got 36%. The medium agent only got 29% but this is probably due to variation. These results show that the NatureCNN feature extractor is good against complex scenes.

### *B. Limitations*

There are a lot of limitations. Each configuration was trained with a single random seed. The differences between the configurations may be because of stochastic variation in the training instead of true complexity. Another limitation is the task is a one step MDP. The agent shoots once per episode and receives a single reward. This simplifies the credit assignment but does not get the decision making that is a part of most real visual RL tasks. Another limitation is that the action space was very simple and removed the relative yaw offset. And the final limitation would be the scenes configured. The gap between the medium and high scene were not very large.

### *C. Future Work*

In the future, I would run each configuration with multiple random seeds. This would allow me to put confidence intervals on the performance comparisons and remove the variance.

I could also do sequential MDP. This is where the agent can adjust its aim over multiple steps before shooting which would test whether the output the agent had holds under harder credit assignment conditions.

Finally, the complexity metric could be made to focus on the actual basketball hoop instead of the entire scene.

## REFERENCES


[1] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[2] A. Zhang, R. McAllister, R. Calandra, Y. Gal, and S. Levine, "Learning invariant representations for reinforcement learning without reconstruction," in *International Conference on Learning Representations (ICLR)*, 2021.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[4] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[5] K. Cobbe, C. Hesse, J. Hilton, and J. Schulman, "Leveraging procedural generation to benchmark reinforcement learning," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020, pp. 2048–2056.

[6] E. Coumans and Y. Bai, "Bullet physics SDK," https://github.com/bulletphysics/bullet3, 2021.